

Methodology: Deep Learning Segmentation & Spatial XR Calibration for Neuro-Oncology Guidance

Medical Vision & Spatial XR Research Group

September 2026

Overview

This document describes the technical approach behind the segmentation and spatial calibration pipeline used for intraoperative neuro-oncology mixed reality guidance. Brain MRI slices are segmented with a U-Net trained on a hybrid Dice-BCE loss, converted to a 3D surface mesh via Marching Cubes, and imported into Unity, where DICOM voxel spacing ($0.686 \times 0.686 \times 5.0$ mm) drives a metric-accurate spatial transform for the XR headset. In testing, the model reached a Dice score of 0.914 ± 0.021 and the XR viewer rendered at 90 FPS with sub-millimeter registration accuracy.

The pipeline has two parts:

1. **Computer vision** — segments MRI slices and reconstructs a 3D surface.
2. **Spatial XR** — calibrates that surface to real-world scale and displays it in the headset.

[MRI slices] -> [U-Net] -> [3D volume] -> [Marching Cubes] -> [.OBJ mesh]
|
v
[Unity: affine scale + calibration] -> [XR headset]

1. Computer Vision Pipeline

Preprocessing. Each slice is min-max normalized:

$$I_{norm}(x, y) = \frac{I(x, y) - \min(I)}{\max(I) - \min(I)}$$

Model. A U-Net with 4 down/up-sampling stages (channels $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$), standard double-conv blocks (Conv $\rightarrow$ BN $\rightarrow$ ReLU, twice) with skip connections between matching encoder/decoder levels.

Loss function. Lesion slices are a minority class (~35% of the dataset), so plain BCE under-trains on positives. We combine BCE with Dice loss:

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$
$$\mathcal{L}_{Dice} = 1 - \frac{2 \sum_i y_i \hat{y}_i + \epsilon}{\sum_i y_i + \sum_i \hat{y}_i + \epsilon}, \quad \mathcal{L}_{Total} = \mathcal{L}_{BCE} + \mathcal{L}_{Dice}$$

($\epsilon = 10^{-6}$, just to avoid divide-by-zero.)

Surface extraction. Predicted 2D masks are stacked along Z into a scalar density field $\Phi(x, y, z) \in [0, 1]$. Marching Cubes extracts the isosurface at $\Phi = 0.5$, and surface normals come from the gradient of Φ . The result is exported as an indexed .obj mesh.

2. Spatial XR Calibration

Why it’s needed. Imported meshes default to arbitrary unit sizes in the game engine — for surgical use they need to match the patient’s real anatomy exactly.

Voxel-to-world scaling. Using the DICOM pixel spacing ($\Delta x = \Delta y = 0.686$ mm, $\Delta z = 5.0$ mm), each mesh vertex is scaled from voxel units into real-world meters:

$$S_{uniform} = \frac{\Delta x}{1000} = 6.86 \times 10^{-4} \text{ m/voxel}$$

Camera setup. Because the anatomy is viewed at near-miniature scale, the default XR camera clips too aggressively. We set the near clip plane to 1 cm and narrow the field of view to $\sim 20^\circ$ for a “microscope” viewing mode.

Interaction. Rotation is smoothed with quaternion SLERP (Quaternion.Slerp) so the model doesn’t jump around as the surgeon orbits it:

$$q(t) = \frac{\sin((1-t)\Omega)}{\sin \Omega} q_0 + \frac{\sin(t\Omega)}{\sin \Omega} q_1, \quad \cos \Omega = q_0 \cdot q_1$$

3. Results

Metric	Value
Dice score (DSC)	0.914 ± 0.021
Mean IoU	0.842 ± 0.018
Inference latency (per slice)	11.4 ms
Mesh density	48,912 polygons
XR render latency	11.1 ms (90 FPS)
Spatial registration accuracy	< 0.2 mm error